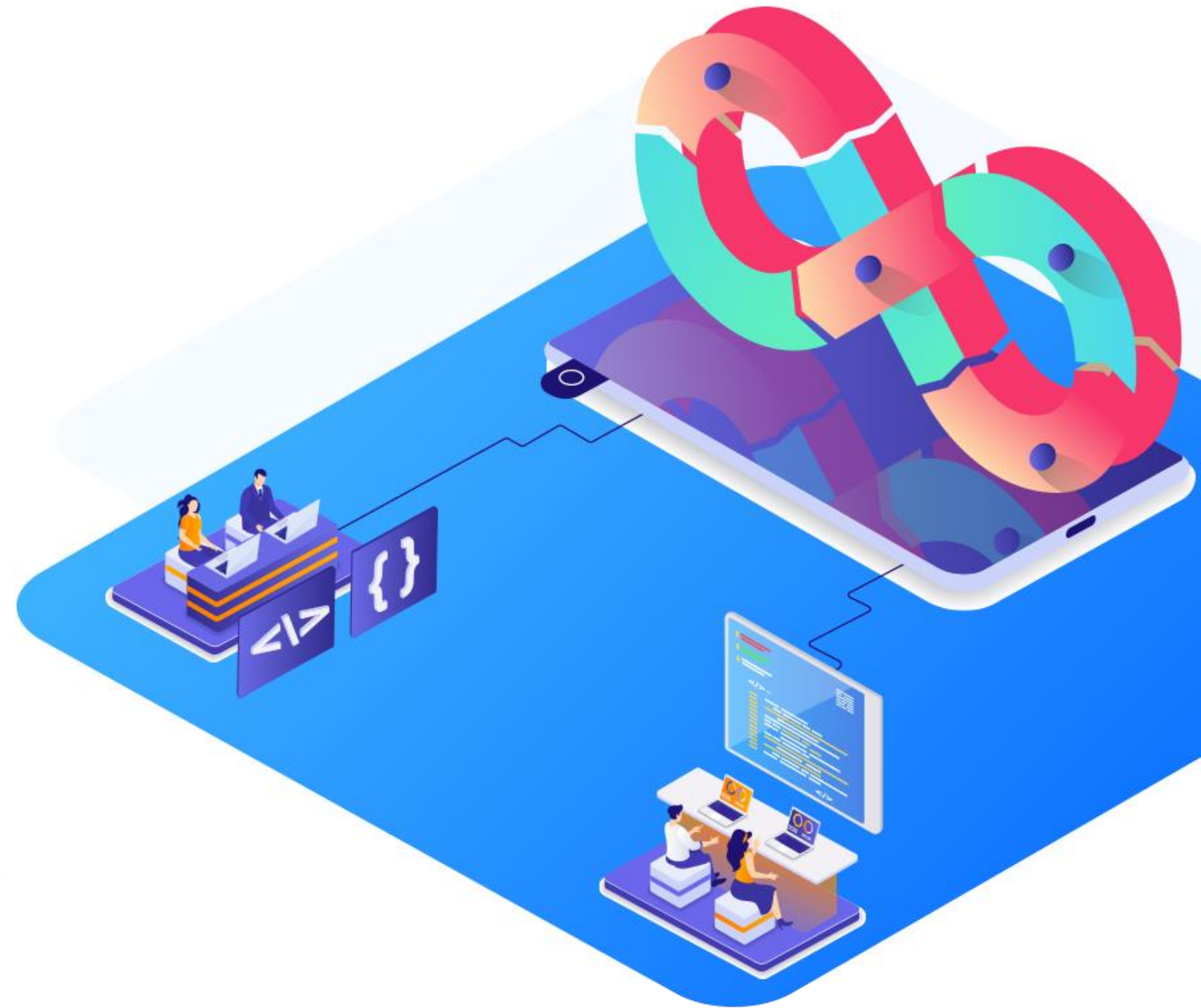


Capstone Project





Implementing microservices with CodePipeline and ECS

Objective

The objective of this project is to showcase the end-to-end process of deploying a microservices-based application with separate Backend (NodeJS) and Frontend (React) services using AWS CodePipeline and Amazon ECS. This project demonstrates the automation of CI/CD workflows, efficient containerization of both backend and frontend applications, and the deployment of these services on a scalable ECS cluster. The setup ensures high availability, seamless updates, and a reliable architecture utilizing serverless containers with AWS Fargate.



Problem Statement and Motivation



Real-time scenario:

DevBank, a rapidly expanding financial technology company, offers a digital banking platform to clients across North America. To enhance scalability and handle increasing user transactions, DevBank is transitioning its backend services (NodeJS API) and frontend interface (React application) to a microservices architecture deployed on AWS.

As a DevOps Engineer at DevBank, you are tasked with implementing automated CI/CD pipelines for both backend and frontend services. This project involves using Docker for containerization, AWS CodePipeline for automating builds, and Amazon ECS with AWS Fargate for deploying the services in a serverless, high-availability setup. Your responsibility is to ensure an efficient, automated deployment process that reduces release time and maintains a seamless user experience, enabling DevBank to scale its platform and deliver reliable services to its growing customer base.

Industry Relevance

The following tools used in this project serve specific purposes within the industry:

1. **AWS CodePipeline:** Automates the build, test, and deployment stages for continuous delivery, ensuring faster and consistent software releases.
2. **AWS CodeBuild:** Compiles the source code and creates Docker images, seamlessly integrating with CodePipeline for automated builds.
3. **Docker:** Enables efficient containerization of the application, packaging the backend and frontend services with their dependencies for consistent deployment.
4. **GitHub:** Acts as the version control system, hosting the source code for both backend and frontend applications, facilitating collaboration and code management.



Business Impact



GitHub:

GitHub has experienced a 50% increase in utilization by teams integrating with cloud-native CI/CD services like AWS CodePipeline, highlighting its role in collaborative development and secure code hosting.

The growth underscores the trend towards centralized version control, improving team collaboration and project transparency.

AWS CodePipeline:

AWS CodePipeline has seen a 60% increase in adoption among enterprises leveraging automated CI/CD workflows to accelerate software delivery.

The rise in usage highlights the industry's shift towards continuous integration and delivery, reducing deployment times and enhancing software quality.

Tasks

The following tasks outline the process of implementing a containerized microservices application with automated CI/CD pipelines using AWS CodePipeline and Amazon ECS:

1. Created and configured Dockerfiles for both Backend NodeJS and Frontend React applications, and committed them to GitHub repositories for version control.
2. Set up the Jenkins environment on the DevOps lab by accessing the pre-installed Jenkins instance, and created a new Jenkins job for automating the CI pipeline.
3. Configured AWS CodePipeline for the backend application, integrating GitHub as the source and AWS CodeBuild for Docker image creation and push to DockerHub.
4. Deployed both backend and frontend services on the ECS Cluster using the configured CodePipelines, and validated the deployments by testing the endpoints with the public IP addresses.



Project References

- **Task 1: Refer to the course:** DevOps on AWS: Lesson 02
- **Task 2: Refer to the course:** DevOps on AWS: Lesson 02
- **Task 3: Refer to the course:** Containerization with Docker: Lesson 04
- **Task 4: Refer to the course:** DevOps on AWS: Lesson 02



Output Screenshots

Interface displays ECS service statuses overview.:

ServicesTasksInfrastructureMetricsScheduled tasksTags

Services (2) Info

Filter services by value

Filter launch typeAny launch type

Filter service typeAny service type

< 1 > ⚙

☐

Service name ▲

ARN

Status ▼

Service t... ▼

Deployments and tasks ▼

☐

[backend-service](#)

arn:aws:ec...

✔ Active

REPLICA

1/1 Tasks running

☐

[frontend-service](#)

arn:aws:ec...

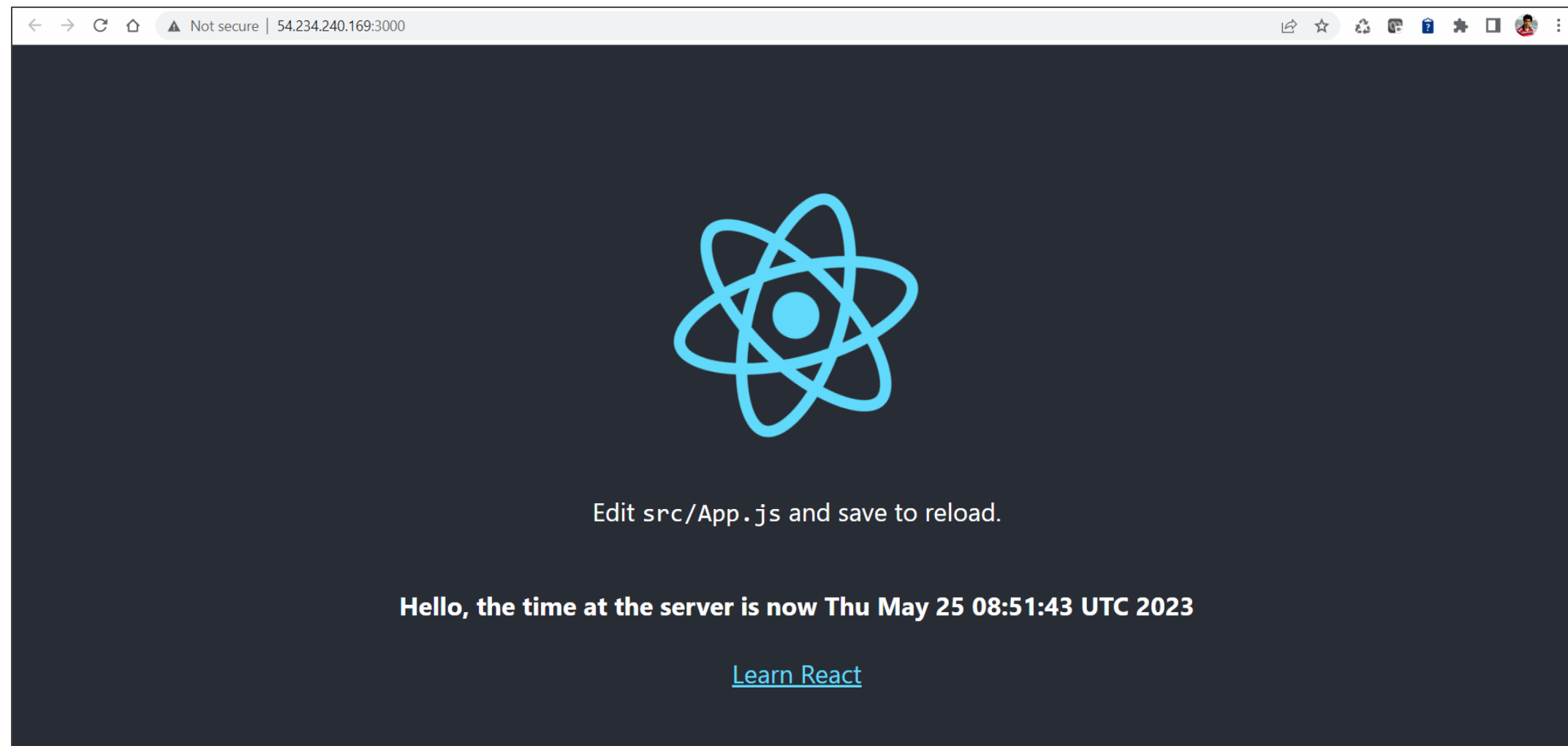
✔ Active

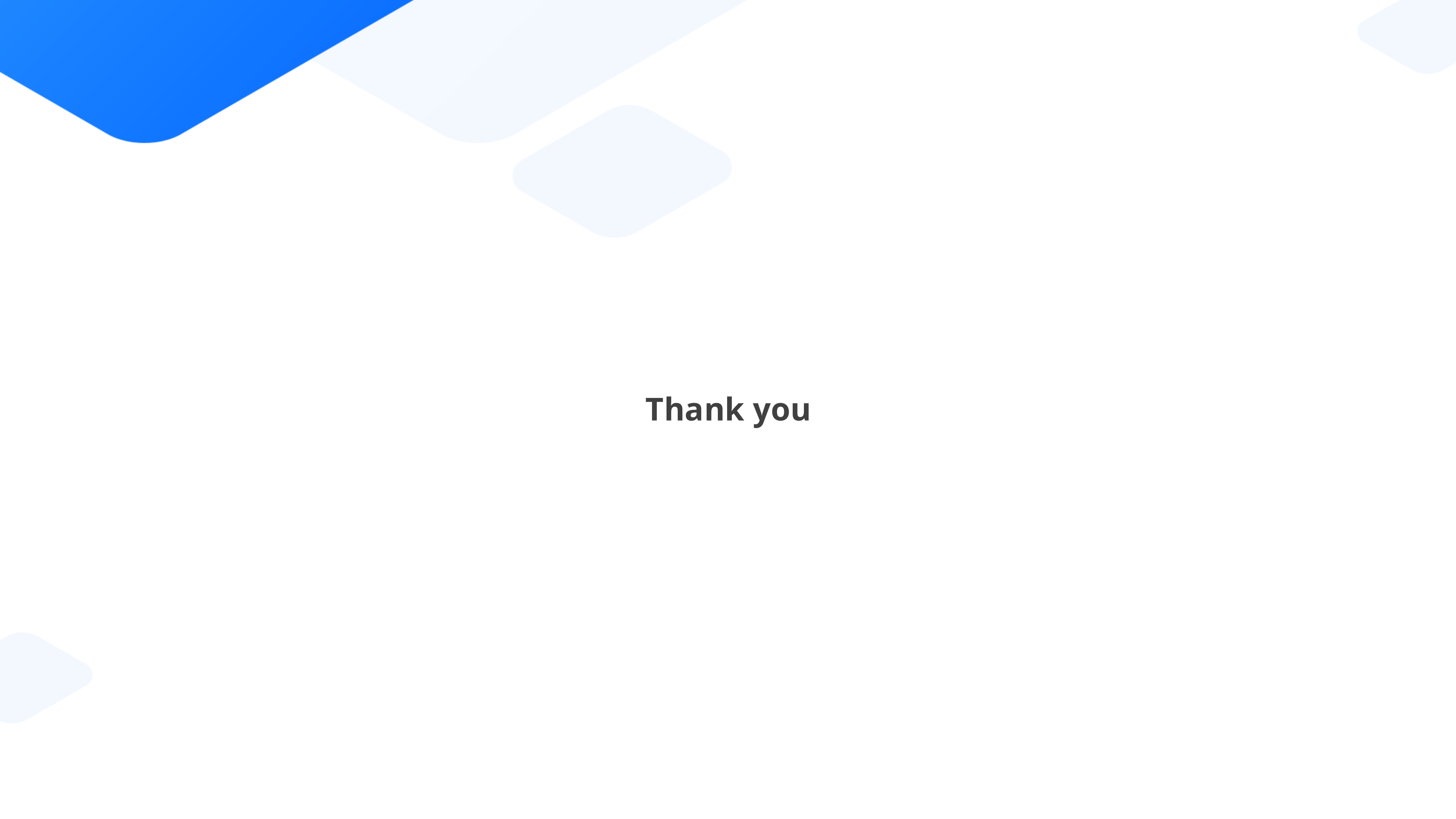
REPLICA

1/1 Tasks running

Output Screenshots

React application running on public IP





Thank you